

Edge cases (200 doesn't mean safe to trust) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

There are two separate problems to demonstrate here, and they need two separate pieces of code.

Problem 1: the response body isn't valid JSON at all — maybe it's an HTML error page. Calling `.json()` on that raises an error, and you should catch it specifically, not let it crash your program.

Problem 2: the body *is* valid JSON, but a key your code expected just isn't there. Python gives you two different ways to read a dictionary key — one that crashes if it's missing, and one that lets you supply a fallback instead.

Things to use:

- `import json` and `json.loads("this is not json")` — raises `json.JSONDecodeError`.
- `try: ... except json.JSONDecodeError as e: print(e)` — catch it and print what went wrong.
- `data = {"a": 1}` then `data.get("missing_key")` — prints `None`, no crash.
- `data["missing_key"]` — raises `KeyError` if you try this instead.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Problem 1 — invalid JSON: `response.json()` internally calls `json.loads()` on the body text and raises `json.JSONDecodeError` (technically `requests.exceptions.JSONDecodeError` when called through `requests`, which subclasses it) if the body can't be parsed at all. You don't need a real broken server to test this — build a fake object, or use `json.loads("not json")` directly.

Problem 2 — valid JSON, wrong shape: once you have a real `dict`, `some_dict["missing_key"]` raises `KeyError` immediately. `some_dict.get("missing_key")` instead returns `None` (or a default you supply: `.get("missing_key", "fallback")`) — no error at all. Neither one is universally "correct" — it depends whether a missing key is a bug you want to know about loudly, or a normal case you want to handle gracefully.

The exact pieces:

- `json.JSONDecodeError` — a subclass of `ValueError`, raised with a `.msg`, `.pos`, and a useful `str(e)` describing exactly where the parsing broke.
- `dict.get(key, default=None)` — never raises; returns `default` if the key isn't present.
- `dict[key]` — raises `KeyError` immediately if the key is missing; this is the "loud" option.
- **Choosing between them isn't random** — use `[key]` (or check with `in` first) for fields your code genuinely can't function without; use `.get(key, default)` for genuinely optional fields where a sensible fallback exists.

Sketch both blocks separately before checking Hint 2.

"Missing key" is the simplest way valid JSON can be the wrong shape, but a real API can also hand you a key that's *present* with a completely different type than expected (`response["choices"]` is a string instead of a list, because of an error path on the server's side). `isinstance(value, list)` (or `dict`, `str`, etc.) — checking the *type* of a value you got back, not just whether the key existed — is worth knowing exists; this exercise focuses on the single-key case, and a genuinely nested shape (several dict/list levels deep) is exactly what Doc04's `chat_client.py` reaches for a `pydantic` model to solve instead of hand-walking.

Difference between Basic and Intermediate: Basic and Intermediate both handle the single-level case — one key, either present with the right type or genuinely missing — which is exactly what this document's Build Task needs.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
part 1 - bad JSON:
    try:
        parse the text "this is not json" as JSON
    except JSON error as e:
        print the error

part 2 - missing key:
    make a dict with one key
    try .get("missing_key") -> see None, no crash
    try ["missing_key"] -> see it crash with KeyError
```

Here is almost the whole thing:

```
# response_validation_practice.py
import json

try:
    json.loads("<html>not json</html>")
except json.JSONDecodeError as e:
    print("bad json:", e)
```

Expected output: bad json: Expecting value: line 1 column 1 (char 0) . What's missing: the missing-key demonstration with both `.get()` and `[...]` . Add it yourself, then check the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
import json

try:
    json.loads("this is not json")
except json.JSONDecodeError as e:
    print(f"invalid JSON: {e}")

data = {"choices": []}
print(data.get("message"))          # None, no crash

try:
    data["message"]
except KeyError as e:
    print(f"missing key: {e}")
```

```
# response_validation_practice.py
import json

try:
    json.loads("<html>not json</html>")
except json.JSONDecodeError as e:
    print(f"bad json: {e}")

data: dict = {"choices": []}
print(data.get("message")) # safe, prints None
```

What's missing: the crashing version of the missing-key lookup, wrapped so it doesn't kill the script. Write it yourself, then compare against the [Solution](#). Notice both branches of problem 2 run — first the safe `.get()`, then, separately, the crashing `[...]` wrapped in its own `try/except` so it doesn't stop the script.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)